OFFICIAL TECHNICAL DOCUMENTATION

EnjoyFreeDeals

Complete Architecture, Database Schema, REST API Reference, Automation Pipelines & Deployment Operations Manual

Project Name	EnjoyFreeDeals (Mobile & Web Platform)
Architecture Version	v1.0.0 (Production Verified)
Database Status	37 / 37 Supabase Tables Healthy & Active (0 Missing)
Stack Components	Android Kotlin, React + Vite Web, Node.js Express API, Supabase PostgreSQL
Database Project Ref	pzgyphnerjatlqlvvvsl (Supabase Cloud)
Documentation Date	October 2026

Table of Contents

1. System Architecture & Ecosystem Section 1

2. Database Architecture & 37 Monitored Tables Section 2

3. Consolidated Migrations & Schema Integrity Section 3

4. Complete REST API Reference Section 4

5. Automation, Scrapers & Telegram Sync Engine Section 5

6. Client Integration (Android, Web & Admin Panel) Section 6

7. Security, Roles & Row Level Security (RLS) Section 7

8. Environment Variables Reference Section 8

9. Deployment & Operations Manual Section 9

> **System Status: All 37 Tables Operational**
>
> All 37 database tables across Catalog, Deals, Pricing, Users, Telegram, and Wallet have passed health checks with 0 missing tables and 0 schema errors.

1. System Architecture & Ecosystem

EnjoyFreeDeals is an end-to-end deal aggregation and price comparison ecosystem consisting of four primary components:

1.1 Core Components

Component	Technology	Role & Responsibilities
Android App	Kotlin, Jetpack Compose / XML, Retrofit, Coroutines	Native mobile app delivering personalized feed, price alerts, notifications, and deal discovery. Consumes backend API.
Web Portal	React 18, Vite, TypeScript, Tailwind CSS, Lucide Icons	Fast web interface for browsing deals, search, store comparisons, blog posts, and administrative management.
Backend API	Node.js 20+, Express.js, Supabase Client SDK	Secure API layer for affiliate link resolution, telegram loot scrapers, deal scoring, price tracking, and admin health checkers.
Database & Auth	Supabase (PostgreSQL 15), RLS, Realtime Replication	Centralized transactional store, user authentication, role-based access control, and automated deal expiration workflows.

1.2 Architectural Security Boundary

> **Critical Security Separation: Public vs. Service Role**
>
> - **SUPABASE_ANON_KEY**: Allowed in Android app and web client for read-only / public operations guarded by PostgreSQL Row Level Security (RLS).
> - **SUPABASE_SERVICE_ROLE_KEY**: STRICTLY restricted to the Node.js backend. Never bundled in Android APKs, web code, or client repositories.
> - Clients communicate through the backend API (`http://localhost:5000` or production Vercel base URL) for administrative and privileged updates.

2. Database Architecture & 37 Monitored Tables

The database layer consists of 37 structured tables divided into functional modules. All tables are actively checked and verified healthy:

Module	Table Name	Status	Purpose & Data Model
Users	`profiles`	OK	User profiles linked to auth.users, avatars, display names.
Users	`user_roles`	OK	Role assignments (e.g., admin, moderator, premium).
Users	`user_preferences`	OK	Category preferences, price alert thresholds, notification options.
Users	`user_activity`	OK	Audit log of user actions, clicks, and session footprints.
Users	`referrals`	OK	Referral reward tracking, inviter/invitee relationships.
Catalog	`categories`	OK	Deal categories (Electronics, Fashion, Grocery, Mobiles, Travel).
Catalog	`stores`	OK	E-commerce store directory (Amazon, Flipkart, Myntra, Ajio, Meesho).
Deals	`deals`	OK	Active deal items, original/deal price, discount, affiliate link, score.
Deals	`products`	OK	Canonical catalog products aggregated across multiple stores.
Deals	`product_offers`	OK	Individual store offers for products with Realtime replication enabled.
Content	`blog_posts`	OK	Editorial deals, promotional articles, shopping guides.
Engagement	`deal_watchlist`	OK	Watchlisted deals flagged by users for price drop updates.
Engagement	`saved_deals`	OK	User bookmark collection of favorite deals.
Engagement	`shared_deals`	OK	Share event tracking across WhatsApp, Telegram, and social links.
Engagement	`deal_votes`	OK	Community upvotes / downvotes for crowd verification.
Engagement	`deal_comments`	OK	User comments and discussions on deal listings.
Engagement	`upvoted_deals`	OK	Quick cache of user upvoted items.
Engagement	`deal_upvotes`	OK	Idempotent unique constraint table for upvote tracking.

Module	Table Name	Status	Purpose & Data Model
Engagement	`recently_viewed_deals`	OK	User browsing history for personalized recommendations.
Pricing	`price_alerts`	OK	Target price threshold triggers set by shoppers.
Pricing	`price_history`	OK	Historical pricing logs for charting product lowest/highest price.
Pricing	`price_comparisons`	OK	Multi-store price matrix (Amazon vs Flipkart vs Croma).
Pricing	`price_comparison_platforms`	OK	Store platform metadata and deep link configuration.
Pricing	`price_tracking_requests`	OK	Async price tracking jobs submitted by users via product URLs.
Pricing	`tracked_products`	OK	Products actively monitored on interval schedules.
Notifications	`notifications`	OK	Push and in-app notifications sent to users.
Analytics	`deal_clicks`	OK	Outbound affiliate click tracking and conversion analytics.
Wallet	`wallets`	OK	User cashback balances and reward earnings.
Wallet	`cashback_transactions`	OK	Credit/debit ledger entries for verified purchases.
Wallet	`withdrawal_requests`	OK	Payout requests (UPI, Bank Transfer, Gift Cards).
Imports	`fetch_runs`	OK	Execution log of automated scrape cycles.
Imports	`scraper_jobs`	OK	Background scraper queue items and processing status.
Imports	`deal_sources`	OK	Configured feed sources, RSS channels, and scrapers.
Imports	`scraped_deal_items`	OK	Staging buffer of raw uncleaned deals before validation.
Telegram	`telegram_channels`	OK	Channels monitored for live loot deals (e.g., India_loot_deals).
Telegram	`scrape_logs`	OK	Telegram message parse logs, error captures, and sync counts.
Telegram	`telegram_sources`	OK	Active Telegram bot and webhook connection metadata.

3. Consolidated Migrations & Schema Integrity

To avoid fragmented SQL migrations, all missing administrative and engagement tables were combined into a single, idempotent migration:

> **Migration File Location**
> `supabase/migrations/20261011_create_all_missing_admin_tables.sql`

3.1 Key Tables Created in the Consolidated Migration

- `products` & `product_offers`: Canonical catalog and multi-store offers with foreign key cascading.
- `fetch_runs`: Tracks run duration, items discovered, items updated, and errors.
- `saved_deals` & `recently_viewed_deals`: Per-user engagement features with composite unique keys.
- `price_alerts`, `price_tracking_requests` & `tracked_products`: Dynamic price drops and scheduler queue.
- `telegram_channels`, `telegram_sources` & `scrape_logs`: Real-time Telegram loot monitoring.
- `upvoted_deals` & `deal_upvotes`: Concurrency-safe community upvote tracking.

3.2 Verification Script

Run the automated database test script at any time to verify health:

```
node scripts/test-tables.js
```

4. Complete REST API Reference

The backend serves JSON REST endpoints under the `/api` prefix.

4.1 System & Health Endpoints

Method	Path	Description
GET	`/api/health`	Returns server uptime, timestamp, memory, and status (ok).
GET	`/api/health/supabase`	Performs live query against Supabase to verify connectivity.
GET	`/api/admin/database/tables`	Inspects all 37 database tables, row counts, and latency.

4.2 Deals & Catalog Endpoints

Method	Path	Description
GET	`/api/deals`	Fetches active deals with optional category, search, and pagination.
GET	`/api/deals/featured`	Returns top-scored trending and hot loot deals.
GET	`/api/deals/:id`	Returns complete deal details, store comparisons, and price history.
POST	`/api/deals/:id/upvote`	Records community upvote and increments upvote_count.

4.3 Price Tracking & Comparison Endpoints

Method	Path	Description
POST	`/api/track-price`	Extracts platform product ID (Amazon ASIN / Flipkart PID) and starts tracking.
GET	`/api/price-history?url=...`	Retrieves 30/90/180-day price trends for chart rendering.

4.4 Telegram Ingestion Endpoints

Method	Path	Description
POST	`/api/telegram-scraper/fetch`	Triggers manual scrape run across configured Telegram public loot channels.

Method	Path	Description
POST	`/api/telegram-scraper/webhook`	Receives real-time Telegram channel message payload from bot webhooks.

5. Automation, Scrapers & Telegram Sync Engine

5.1 Scraper Architecture

The deal ingestion engine automatically scrapes, parses, normalizes, and ranks deals from Telegram channels and merchant feeds.

Script / Task	Schedule	Functionality
`scripts/sync-genie-loot.js`	Every 15-30 mins	Scrapes public preview pages of top Telegram channels, parses titles, discount percentages, and original prices.
`scripts/sync-price-tracking.js`	Hourly	Refreshes prices for all items in tracked_products and logs changes to price_history.
`.github/workflows/telegram-sync.yml`	Cron Schedule	Automated GitHub Actions workflow triggering serverless scrape runs.
`cleanup_expired_deals()`	Every 1 min (pg_cron)	Hides expired deals, mark zero-stock items as inactive.

5.2 URL Normalization & Platform Detection

Links are cleaned to strip tracking pixels and extract standard affiliate parameters:

- **Amazon**: Standardizes `amzn.to` shortlinks to canonical `amazon.in/dp/{ASIN}` and appends associate tag.
- **Flipkart**: Extracts `pid` query parameters and generates standardized affiliate links.
- **Other Platforms**: Detects Myntra, Ajio, Meesho, Croma, Nykaa, and Tata Cliq automatically.

6. Client Integration (Android, Web & Admin Panel)

6.1 Android Client Configuration

In the Android project, configure `local.properties`:

```
SUPABASE_URL=https://pzgyphnerjatlqlvvvsl.supabase.co
SUPABASE_ANON_KEY=<YOUR_PUBLIC_ANON_KEY>
BACKEND_BASE_URL=http://10.0.2.2:5000
```

> **Android Networking Note**
> When testing in Android Studio Emulator, use `http://10.0.2.2:5000` to reach the host machine backend.

6.2 Web Frontend Configuration

The web client runs on Vite with React 18 and Shadcn UI. The development server proxies `/api` requests directly to the Node backend:

```
// web/vite.config.ts
server: {
  port: 8080,
  proxy: {
    '/api': {
      target: 'http://localhost:5000',
      changeOrigin: true
    }
  }
}
```

6.3 Admin Panel Access

- **URL**: `http://localhost:8080/admin` (or `/admin/database`)
- **Admin Secret Key**: `enjoyfreedeals_admin_secret`
- **Features**: Real-time 37-table status monitor, row counts, missing table alerts, and direct SQL migration viewer.

7. Environment Variables Reference

Reference of all environment variables configured in `.env`:

Variable	Required	Description
`SUPABASE_URL`	Yes	Supabase API URL (e.g. `https://pzgyphnerjatlqlvvvsl.supabase.co`).
`SUPABASE_ANON_KEY`	Yes	Public anonymous client key for public read operations.
`SUPABASE_SERVICE_ROLE_KEY`	Yes	Privileged server key for backend admin operations.
`SUPABASE_PROJECT_ID`	Yes	Supabase unique project reference (pzgyphnerjatlqlvvvsl).
`PORT`	No	API server listening port (Default: 5000).
`ADMIN_AUTH_SECRET`	Yes	Admin authentication secret token (enjoyfreedeals_admin_secret).
`TELEGRAM_IMPORT_SECRET`	Yes	Bearer token protecting scraper manual triggers and webhook ingestions.
`GENIE_LOOT_PAGE_URLS`	No	Comma-separated list of Telegram channel preview URLs to scrape.

8. Deployment & Operations Manual

8.1 Production Deployment on Vercel

The repository includes a serverless adapter in `api/index.js` configured via `vercel.json`:

```
# Deploy to production
npm run vercel:deploy
```

8.2 Production Deployment on Linux VPS with PM2

```
# Start process manager
npm run pm2:start

# Check process health
pm2 status

# Restart if updated
npm run pm2:restart
```

8.3 Automated Test Suite

Verify system integrity before deploying:

```
# Run all 39 backend tests
npm test

# Run database health check
node scripts/test-tables.js
```

> **Documentation Verified**
>
> EnjoyFreeDeals backend, database schema, client integration, and scraping pipelines are fully operational.